

# Strategy e State

## Padrões de Projeto - Aula #08

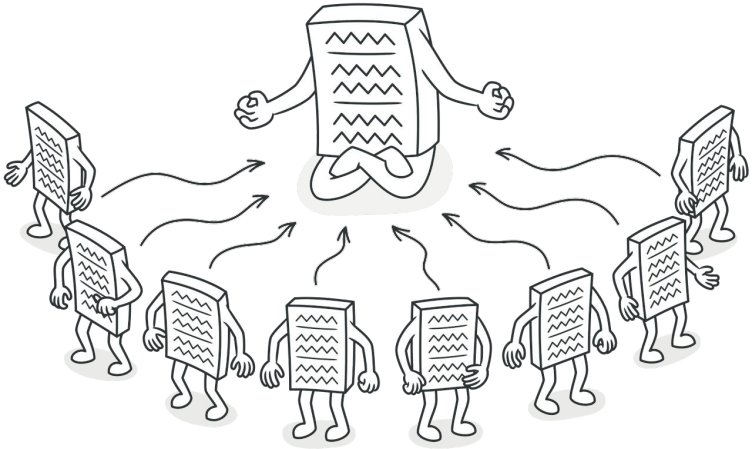
Prof. Dr. Carlos Melo

carlos.melo@upe.br

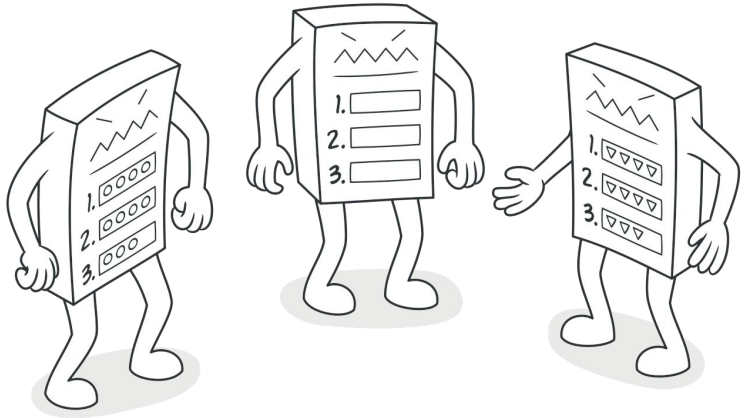
UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

# Padrões Comportamentais

**Strategy**
Família de algoritmos intercambiáveis, escolhidos em tempo de execução



**State**
O objeto muda de comportamento quando seu estado interno muda




- Os dois têm diagramas parecidos, o que muda é a intenção.

# Objetivos de aprendizagem



- Implementar *strategies* e saber quando uma função basta no lugar de uma classe
- Distinguir Strategy de Template Method e Bridge
- Identificar uma máquina de estados escondida em ifs de algum atributo
- Implementa State e transition (algo que o Strategy não tem)

# Strategy - O Problema

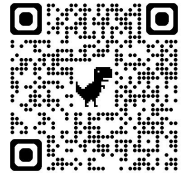


- Temos o UPEMaps que é um app de navegação.
- A primeira versão só traçava rotas de carro.

```python
class Navegador:
    def construir_rota(self, origem, destino):
        # 100+ linhas de algum algoritmo de menor caminho em grafos
        return ["seguir BR-232", "converter à direita", "chegou"]
```

- Novos requisitos:
  - rotas a pé
  - rotas de transporte público
  - rotas de bicicleta
  - rotas turísticas passando por pontos de interesse

# Strategy - O Problema



```python
class Navegador:
    def construir_rota(self, origem, destino, modal):
        if modal == "carro":
            ...   # 100+ linhas
        elif modal == "pe":
            ...   # 100+ linhas
        elif modal == "publico":
            ...   # 100+ linhas
        elif modal == "bicicleta":
            ...   # 100+ linhas
```

- A classe cresceu 300%
  - Qualquer alteração de algoritmo pode demandar alterar uma classe compartilhada por todos os meios de transporte e ocasionar conflitos na hora de algum merge no github.

# Strategy - O Problema



- Usar `if` e `else` não é o problema, mas sim o fato de todos os algoritmos estarem armazenados na mesma classe.
  - Se a classe cresce a cada requisito é sinal de que estamos violando o **O** do S**O**LID, uma vez que cada extensão vai exigir edição.
  - A classe tem quatro responsabilidades, o que nos leva a viola o **S** do **S**OLID.
  - Testar uma rota a pé, requer carregar um objeto `Navegador` com todos os algoritmos.
  - Todo mundo vai editar o mesmo arquivo.
- Além disso, os quatro blocos têm algo em comum:
  - `rota = algoritmo(origem, destino)`
  - Mesma assinatura, mesma finalidade, diferentes implementações.
  - A isso chamamos de família de algoritmos intercambiáveis

# Strategy - Intenção

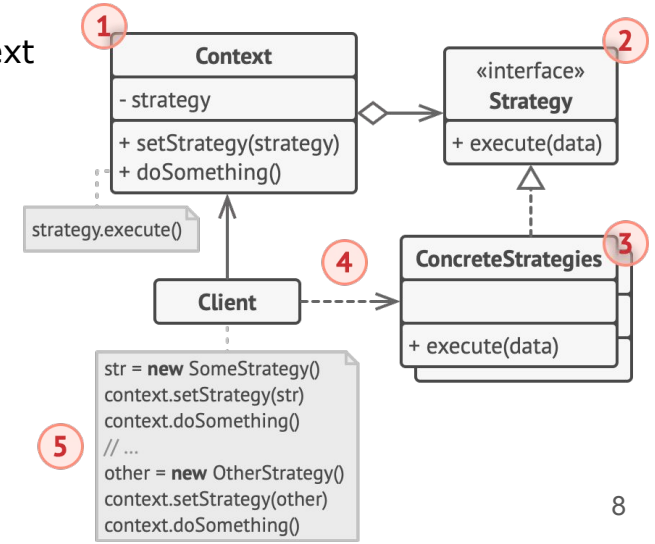


"Definir uma **família** de algoritmos, **encapsular** cada um deles e torná-los **intercambiáveis** . Strategy permite que o algoritmo varie independentemente dos clientes que o utilizam."

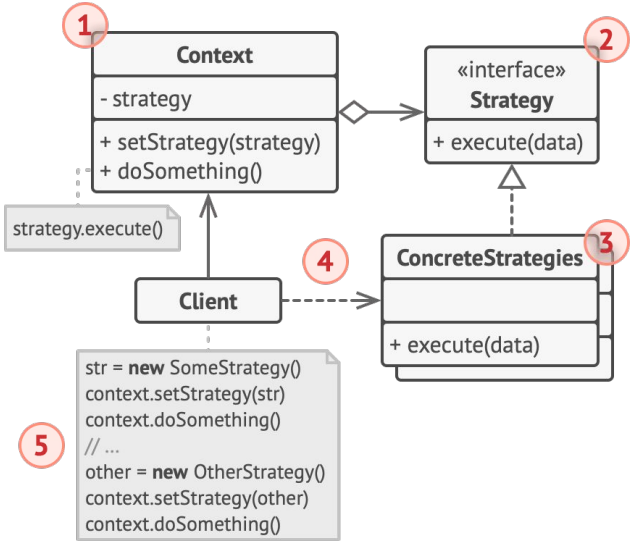
- Padrão comportamental
- Família:
  - Algoritmos que fazem a mesma coisa de formas diferentes
- Encapsular:
  - Cada um em sua própria classe
- Intercambiáveis:
  - O contexto pode ser trocado sem a necessidade de saber qual é qual

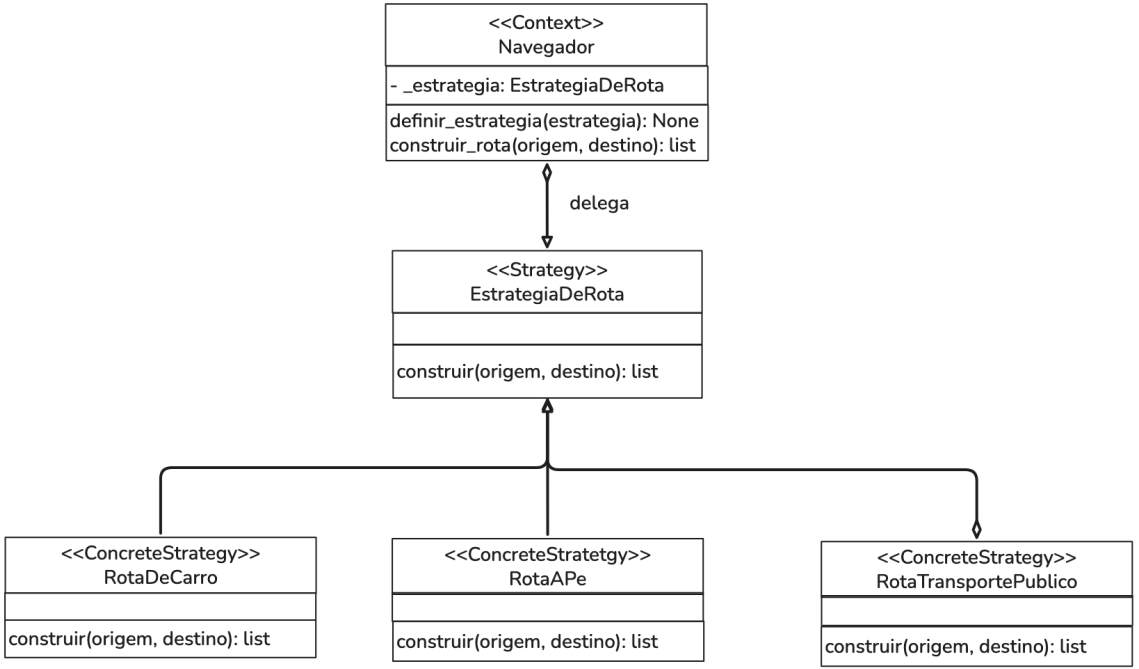
# Strategy - Estrutura

1. **Context** mantém uma referência para uma das **ConcreteStrategies** e se comunica com esse objeto através da interface da estratégia
2. A interface da **Strategy** é comum a todas as **ConcreteStrategies**.
3. **ConcreteStrategies** implementam diferentes variações de um algoritmo que **Context** usa.
4. O Context chama o método de execução no objeto strategy ligado cada vez que ele precisa rodar um algoritmo. O contexto não sabe qual tipo de estratégia ele está trabalhando ou como o algoritmo é executado.
5. **Client** cria um objeto Strategy específico e o repassa para o Context



# Aplicando ao nosso contexto



# Aplicando ao nosso contexto



```python
from abc import ABC, abstractmethod

class EstrategiaDeRota(ABC):
    @abstractmethod
    def construir(self, origem: str, destino: str) -> list[str]: ...


class RotaDeCarro(EstrategiaDeRota):
    def construir(self, origem, destino):
        return [f"Saia de {origem} pela BR-232",
                "Siga por 230 km",
                f"Chegou em {destino} — 3h10, 18 L de combustível"]

class RotaAPe(EstrategiaDeRota):
    def construir(self, origem, destino):
        return [f"Saia de {origem} pela calçada à direita",
                "Atravesse a praça central",
                f"Chegou em {destino} — 47h, evite trechos sem acostamento"]

class RotaTransportePublico(EstrategiaDeRota):
    def construir(self, origem, destino):
        return [f"Terminal de {origem}: ônibus expresso 14:30",
                "Baldeação em Caruaru",
                f"Chegou em {destino} — 4h20, R$ 62,00"]
```

carlos.melo@upe.br

# Versão Pythonica do Strategy

- A interface tem um método. Em Python é só uma função.

```python
from typing import Callable

Estrategia = Callable[[str, str], list[str]]

def rota_de_carro(origem, destino):
    return [f"Saia de {origem} pela BR-232", "..."]

def rota_a_pe(origem, destino):
    return [f"Saia de {origem} pela calçada à direita", "..."]


class Navegador:
    def __init__(self, estrategia: Estrategia):
        self._estrategia = estrategia

    def construir_rota(self, origem, destino):
        for passo in self._estrategia(origem, destino):
            print(passo)
```

# Versão Pythonica do Strategy

- Use classes quando a estratégia tem estado ou mais de uma operação.
  - Caso contrário, função.
- O partial serve para criar uma nova função fixando antecipadamente alguns valores para os argumentos de uma função já existente.

```
1 Navegador(rota_de_carro).construir_rota("Garanhuns", "Recife")
2
3
4 from functools import partial
5 rota_turistica = partial(rota_de_carro, evitar_pedagio=True)
6 Navegador(rota_turistica).construir_rota("Garanhuns", "Recife")
7
8
9 MODAIS = {"carro": rota_de_carro, "pe": rota_a_pe}
10 Navegador(MODAIS[escolha_do_usuario]).construir_rota(origem, destino)
```

# Prós e Contras do Strategy



**Prós**

- Troca de algoritmo em tempo de execução
- Isola a implementação do algoritmo do código que o usa
- Prioriza composição
- Novas estratégias entram sem tocar no contexto (S**O**LID)
- Cada algoritmo pode ser testado individualmente sem tocar no contexto

**Contras**

- Se você tem dois algoritmos que nunca mudam, o padrão é complicação adicional
- O cliente precisa conhecer a estratégia para escolher uma
- Muitas classes pequenas para pouco código

**Não use**:

- Se tiver um único algoritmo ou quando não precisa realizar a troca em tempo de execução. Muitas vezes o if e else basta.

# Template Method - O Problema

- Temos uma aplicação para mineração de dados em documentos corporativos.
- Começamos com arquivos DOC. Depois CSV e PDF.

- Só os dois primeiros passos variam. O processo e a análise são idênticos.
- O cliente se torna apenas um condicional

```python
if caminho.endswith(".doc"): minerador = MineradorDOC()
elif caminho.endswith(".csv"): minerador = MineradorCSV()
elif caminho.endswith(".pdf"): minerador = MineradorPDF()
```

```python
class MineradorDOC:
    def minerar(self, caminho):
        arquivo = self._abrir_doc(caminho)
        bruto = self._extrair_doc(arquivo)
        dados = self._parsear(bruto)
        analise = self._analisar(dados)
        self._enviar_relatorio(analise)
        arquivo.close()
        return analise

class MineradorCSV:
    def minerar(self, caminho):
        arquivo = self._abrir_csv(caminho)
        bruto = self._extrair_csv(arquivo)
        dados = self._parsear(bruto)
        analise = self._analisar(dados)
        self._enviar_relatorio(analise)
        arquivo.close()
        return analise

class MineradorPDF:
    def minerar(self, caminho):
        arquivo = self._abrir_pdf(caminho)
        ...
```

# Template Method - A ideia



- Quebre o algoritmo em passos, escreva a sequência uma vez na classe base, e deixe as subclasses redefinirem só os passos que diferem.

```python
class MineradorDeDados(ABC):
    def minerar(self, caminho): # template method
        arquivo = self._abrir(caminho)
        bruto   = self._extrair(arquivo)
        dados   = self._parsear(bruto)
        analise = self._analisar(dados)
        self._enviar_relatorio(analise)
        self._fechar(arquivo)
        return analise
```

# Template Method - Intenção

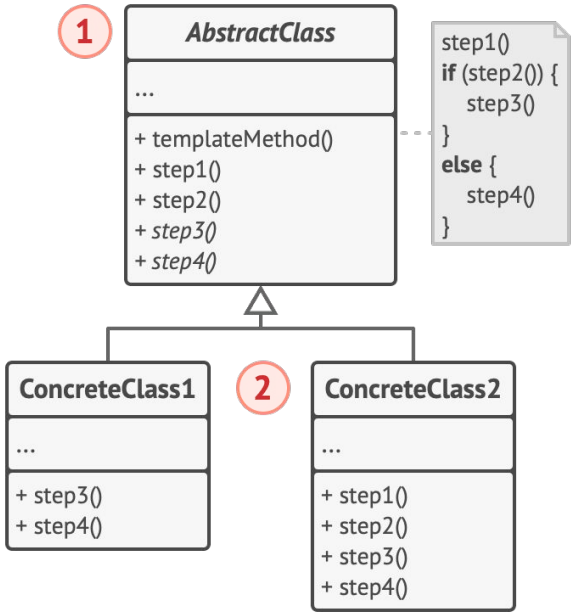


"Definir o **esqueleto** de um algoritmo em uma operação, postergando a definição de alguns passos para subclasses. O Template Method permite que subclasses redefinam certos passos de um algoritmo sem mudar sua estrutura."

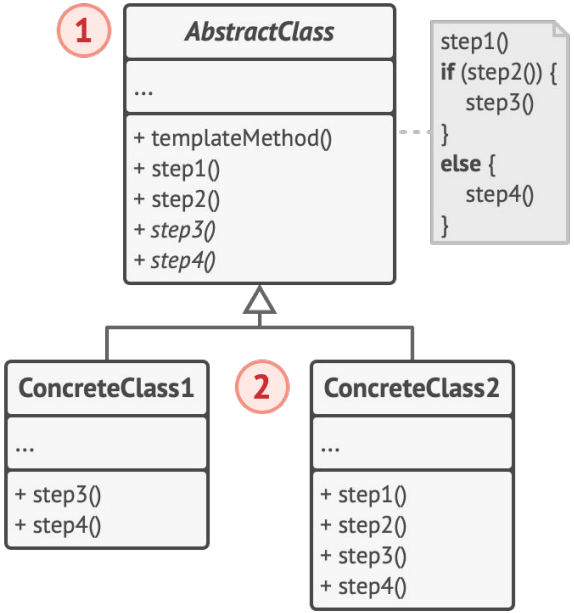
- Padrão comportamental
- Esqueleto é a base que controla a ordem
- As subclasses controlam o conteúdo de passos específicos

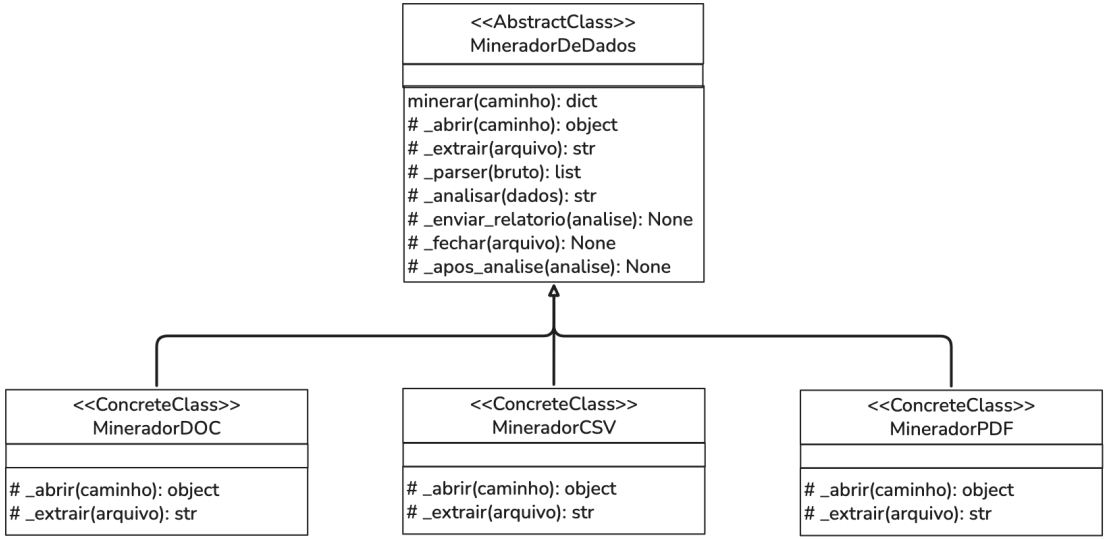
# Template Method - Estrutura

1. **AbstractClass** declara métodos que agem como etapas de um algoritmo, bem como o próprio método padrão *(template)* que chama esses métodos em uma ordem específica.
2. **ConcreteClasses** podem sobrescrever todas as etapas, exceto o método padrão.

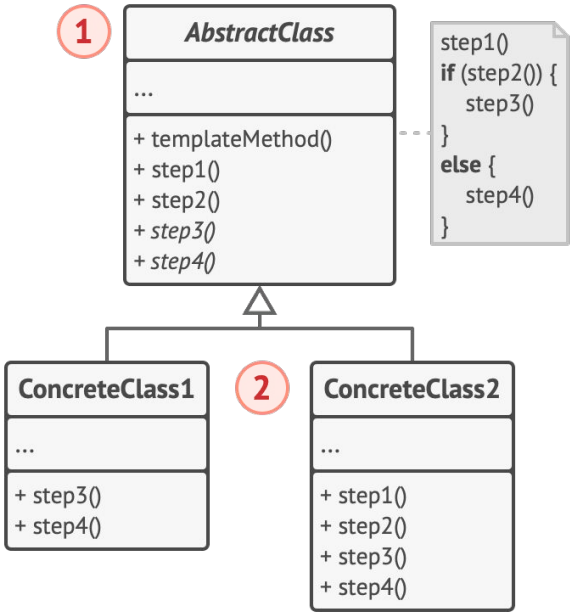


# Aplicando ao nosso domínio



# Aplicando ao nosso domínio




```python
from abc import ABC, abstractmethod

class MineradorDeDados(ABC):
    def minerar(self, caminho: str) -> dict:
        print(f"Minerando {caminho}")
        arquivo = self._abrir(caminho)
        bruto   = self._extrair(arquivo)
        dados   = self._parsear(bruto)
        analise = self._analisar(dados)
        self._apos_analise(analise)
        self._enviar_relatorio(analise)
        self._fechar(arquivo)
        return analise

    @abstractmethod
    def _abrir(self, caminho: str): ...

    @abstractmethod
    def _extrair(self, arquivo) -> str: ...

    def _parsear(self, bruto: str) -> list:
        return [linha for linha in bruto.split(";") if linha]

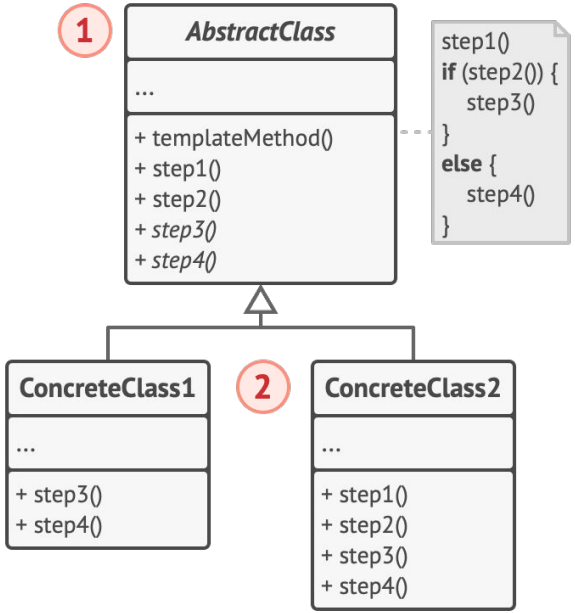
    def _analisar(self, dados: list) -> dict:
        return {"registros": len(dados), "confianca": 1.0}

    def _apos_analise(self, analise: dict) -> None:
        pass

    def _enviar_relatorio(self, analise: dict) -> None:
        print(f"  [REL] {analise['registros']} registros analisados")

    def _fechar(self, arquivo) -> None:
        print("  [IO] arquivo fechado\n")
```

# Aplicando ao nosso domínio




```python
class MineradorCSV(MineradorDeDados):
    def _abrir(self, caminho):
        print("  [IO] abrindo em modo texto")
        return f"<csv: {caminho}>"
    def _extrair(self, arquivo):
        return "linha1;linha2;linha3"

class MineradorPDF(MineradorDeDados):
    def _abrir(self, caminho):
        print("  [IO] abrindo em modo binário")
        return f"<pdf: {caminho}>"
    def _extrair(self, arquivo):
        print("  [OCR] extraindo texto")
        return "pagina1;pagina2"
    def _analisar(self, dados):
        return {"registros": len(dados), "confianca": 0.87}
    def _apos_analise(self, analise):
        print(f"  [OCR] confiança média: {analise['confianca']:.0%}")
```

# Aplicando ao nosso domínio



```python
for m in (MineradorCSV(), MineradorPDF()):
    m.minerar("relatorio_q3")


# Minerando relatorio_q3
#  [IO] abrindo em modo texto
#  [REL] 3 registros analisados
#  [IO] arquivo fechado

# Minerando relatorio_q3
#  [IO] abrindo em modo binário
#  [OCR] extraindo texto
#  [OCR] confiança média: 87%
#  [REL] 2 registros analisados
#  [IO] arquivo fechado
```

# Prós e Contras do Template Method



**Prós**

- Elimina duplicação, uma vez que o algoritmo mora num lugar só
- O cliente pode variar partes de um algoritmo grande.
- Hooks dão pontos de extensão sem exigir que a subclasse saiba da estrutura.
- É a base de alguns frameworks como unittest, Django, Scrapy spiders.
- Pode ser mais rápido que construir, quando há cálculo caro envolvido.

**Contras**

- Baseado em herança então a subclasse fica presa à classe mãe.
- Viola o **L** do SO**L**ID caso uma subclasse suprima algo da classe mãe de modo incompatível.
- É mais difícil mudar em tempo de execução.

**Não use**:

- Se os algoritmos forem muito grandes ou se a classe mãe for se tornar muito extensa.



Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS